ユーザー管理API仕様書

概要

バージョン: 1.0.0
最終更新: 2026-02-20
担当: Development Team

目的

システム管理者がユーザー（組織利用者）を管理するためのAPIを提供します。

対象ユーザー

対象ユーザー

- システム管理者（System Admin）: 全機能へのアクセス

認証・認可

ミドルウェアチェーン

すべてのエンドポイントで以下のミドルウェアチェーンを適用：

```
router.METHOD('/path',
  authMiddleware,           // 1. JWT認証（セッション検証、req.userに格納）
  handleValidationErrors,   // 2. バリデーションエラーハンドリング
  userController.METHOD     // 3. コントローラー
);
```

権限

- **System Admin**: すべての操作が可能

マルチテナント分離

- すべてのDBクエリに `organization_id` フィルタ必須
- authMiddlewareで `req.user.organizationId` を取得
- 他組織のユーザーは閲覧・操作不可

API一覧

No	HTTPメソッド	エンドポイント	説明	権限
1	GET	/api/v1/systemManagement/user	ユーザー一覧取得	System Admin
2	GET	/api/v1/systemManagement/user/:userId	ユーザー詳細取得	System Admin
3	POST	/api/v1/systemManagement/user	ユーザー新規作成	System Admin
4	PUT	/api/v1/systemManagement/user/:userId	ユーザー更新	System Admin
5	PATCH	/api/v1/systemManagement/user/:userId/status	アクティブ状態更新	System Admin
6	GET	/api/v1/systemManagement/user/download-csv	CSVダウンロード	System Admin
7	POST	/api/v1/systemManagement/user/upload-csv	CSVアップロード	System Admin

---

API詳細

1. ユーザー一覧取得

エンドポイント

```
GET /api/v1/systemManagement/user
```

説明

組織内のユーザー一覧を取得します。

リクエスト

クエリパラメータ

パラメータ	型	必須	説明	デフォルト
search	string	✗	検索キーワード（ユーザー名、メールアドレス）	-
is_active	boolean	✗	アクティブ状態フィルター	-
department_cd	string	✗	部署CDフィルター	-
role_cd	string	✗	ロールCDフィルター	-
sort_by	string	✗	ソートキー（user_id, user_name, update_at）	user_id
sort_order	string	✗	ソート順（asc, desc）	asc

レスポンス

成功時（200 OK）


```
{
  "success": true,
  "data": [
    {
      "userId": "user001@example.com",
      "userName": "山田太郎",
      "mailAddress": "user001@example.com",
      "departmentCd": "DEPT001",
      "departmentName": "開発部",
      "roleCd": "ROLE002",
      "roleName": "システム管理者",
      "twoFactorAuth": true,
      "isActive": true,
      "lastLoginAt": "2026-02-24T10:30:00Z",
      "createAt": "2026-01-01T00:00:00Z",
      "updateAt": "2026-02-20T00:00:00Z"
    }
  ]
}
```

エラー時

ステータスコード	エラーコード	説明
401	AUTH_TOKEN_INVALID	認証トークンが無効
403	AUTH_INSUFFICIENT_PERMISSIONS	権限不足
500	SYSTEM_DATABASE_ERROR	データベースエラー

実装メモ

データベーステーブル

- organization_user（主テーブル）
- department（JOIN: 部署名取得）
- role（JOIN: ロール名取得）
- user_authentication（JOIN: 二段階認証フラグ取得）
- login_history（JOIN: 最終ログイン日時取得）

SQLクエリ例

```
SELECT
  ou.user_id,
  ou.user_name,
  ou.mail_address,
  ou.department_cd,
  d.department_name,
  ou.role_cd,
  r.role_name,
  ua.entry_flg AS two_factor_auth,
  ou.is_active,
  lh.last_login AS last_login_at,
  ou.create_at,
  ou.update_at
FROM organization_user ou
LEFT JOIN department d
  ON ou.organization_id = d.organization_id
  AND ou.department_cd = d.department_cd
LEFT JOIN role r
  ON ou.organization_id = r.organization_id
  AND ou.role_cd = r.role_cd
LEFT JOIN user_authentication ua
  ON ou.user_id = ua.user_id
  AND ou.organization_id = ua.organization_id
LEFT JOIN (
```

```
  SELECT user_id, organization_id, MAX(login_at) as last_login
  FROM login_history
  WHERE success = true
  GROUP BY user_id, organization_id
) lh
  ON ou.user_id = lh.user_id
  AND ou.organization_id = lh.organization_id
WHERE ou.organization_id = $1
ORDER BY ou.user_id ASC;
```

---

2. ユーザー詳細取得

エンドポイント

```
GET /api/v1/systemManagement/user/:userId
```

説明

指定したユーザーの詳細情報を取得します。

リクエスト

パスパラメータ

パラメータ	型	必須	説明
userId	string	○	ユーザーID（メールアドレス）

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "userId": "user001@example.com",
    "userName": "山田太郎",
    "mailAddress": "user001@example.com",
    "departmentCd": "DEPT001",
    "departmentName": "開発部",
```

```
    "roleCd": "ROLE002",
    "roleName": "システム管理者",
    "twoFactorAuth": true,
    "isActive": true,
    "createAt": "2026-01-01T00:00:00Z",
    "updateAt": "2026-02-20T00:00:00Z"
  }
}
```


エラー時

ステータスコード	エラーコード	説明
404	RESOURCE_NOT_FOUND	ユーザーが存在しない

---

3. ユーザー新規作成

エンドポイント

```
POST /api/v1/systemManagement/user
```

説明

新しいユーザーを作成します。

リクエスト

リクエストボディ


```
{
  "userId": "newuser@example.com",
  "userName": "新規ユーザー",
  "mailAddress": "newuser@example.com",
  "departmentCd": "DEPT001",
  "roleCd": "ROLE003",
  "twoFactorAuth": false,
  "isActive": true,
  "password": "TempPassword123!"
}
```

フィールド	型	必須	説明
userId	string	○	ユーザーID（メールアドレス形式）
userName	string	○	ユーザー名（最大30文字）
mailAddress	string	○	メールアドレス
departmentCd	string	✗	部署CD
roleCd	string	○	ロールCD
twoFactorAuth	boolean	✗	二段階認証設定（デフォルト: false）
isActive	boolean	✗	アクティブ状態（デフォルト: true）
password	string	○	初期パスワード（8文字以上）

レスポンス

成功時（201 Created）


```
{
  "success": true,
  "data": {
    "userId": "newuser@example.com",
    "userName": "新規ユーザー",
    "mailAddress": "newuser@example.com",
    "departmentCd": "DEPT001",
    "departmentName": "開発部",
    "roleCd": "ROLE003",
    "roleName": "実験管理者",
    "twoFactorAuth": false,
    "isActive": true,
    "createAt": "2026-02-20T08:48:00Z",
    "updateAt": "2026-02-20T08:48:00Z"
  },
  "message": "ユーザーを作成しました"
}
```


エラー時

ステータスコード	エラーコード	説明
400	VALIDATION_ERROR	バリデーションエラー
409	RESOURCE_ALREADY_EXISTS	ユーザーIDが既に存在

実装メモ

トランザクション処理

1. organization_userテーブルにINSERT
2. user_authenticationテーブルにINSERT（パスワードハッシュ化）

---

4. ユーザー更新

エンドポイント

```
PUT /api/v1/systemManagement/user/:userId
```

説明

ユーザー情報を更新します。

リクエスト

パスパラメータ

パラメータ	型	必須	説明
userId	string	○	ユーザーID（メールアドレス）

リクエストボディ

```
{
  "userName": "山田太郎（更新）",
  "departmentCd": "DEPT002",
  "roleCd": "ROLE002"
}
```

フィールド	型	必須	説明
userName	string	✗	ユーザー名
departmentCd	string	✗	部署CD

roleCd	string	✗	ロールCD

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "userId": "user001@example.com",
    "userName": "山田太郎（更新）",
    "mailAddress": "user001@example.com",
    "departmentCd": "DEPT002",
    "departmentName": "営業部",
    "roleCd": "ROLE002",
    "roleName": "システム管理者",
    "twoFactorAuth": true,
    "isActive": true,
    "updateAt": "2026-02-20T08:50:00Z"
  },
  "message": "ユーザー情報を更新しました"
}
```

---

5. アクティブ状態更新

エンドポイント

```
PATCH /api/v1/systemManagement/user/:userId/status
```

説明

ユーザーのアクティブ状態を更新します。

リクエスト

パスパラメータ

パラメータ	型	必須	説明

userId	string	○	ユーザーID

リクエストボディ

```
{
  "isActive": false
}
```

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "userId": "user001@example.com",
    "isActive": false
  },
  "message": "ユーザーのアクティブ状態を更新しました"
}
```

---

6. CSVダウンロード

エンドポイント

```
GET /api/v1/systemManagement/user/download-csv
```

説明

ユーザー一覧をCSV形式でダウンロードします。

リクエスト

クエリパラメータ: API #1と同じフィルターパラメータ

レスポンス

成功時（200 OK）

Content-Type: `text/csv; charset=utf-8`

```
ユーザーID,ユーザー名,メールアドレス,所属部署CD,所属部署名,ロールCD,ロール名,二段階認証,アクティブ,更新日時
user001@example.com,山田太郎,user001@example.com,DEPT001,開発部,ROLE002,システム管理者,済,true,2026/02/20 08:48
```

---

7. CSVアップロード

エンドポイント

```
POST /api/v1/systemManagement/user/upload-csv
```

説明

CSV形式でユーザーを一括登録します。

リクエスト

Content-Type: `multipart/form-data`

フォームデータ

フィールド	型	必須	説明
file	File	○	CSVファイル（最大10MB）

CSVフォーマット

```
ユーザーID,ユーザー名,メールアドレス,所属部署CD,ロールCD,二段階認証,アクティブ,初期パスワード
newuser1@example.com,新規ユーザー1,newuser1@example.com,DEPT001,ROLE003,未,true,Password123!
newuser2@example.com,新規ユーザー2,newuser2@example.com,DEPT002,ROLE004,未,true,Password456!
```

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "total": 10,
    "success": 8,
    "failed": 2,
    "errors": [
      {
        "row": 3,
        "userId": "duplicate@example.com",
        "error": "ユーザーIDが既に存在します"
      }
    ]
  },
  "message": "8件のユーザーを登録しました（2件失敗）"
}
```

エラーコード一覧

エラーコード	HTTPステータス	説明
VALIDATION_ERROR	400	バリデーションエラー
AUTH_TOKEN_INVALID	401	認証トークンが無効
AUTH_INSUFFICIENT_PERMISSIONS	403	権限不足
RESOURCE_NOT_FOUND	404	リソースが存在しない
RESOURCE_ALREADY_EXISTS	409	リソースが既に存在
SYSTEM_DATABASE_ERROR	500	データベースエラー

セキュリティ考慮事項

パスワード管理

- パスワードはbcryptでハッシュ化して保存

- API応答にパスワード情報を含めない
- 初期パスワードは8文字以上必須

マルチテナント分離

- すべてのクエリに `organization_id` フィルタを適用
- 他組織のユーザーデータへのアクセスを厳禁

監査ログ

- ユーザー作成・更新・削除操作をログに記録
- create_user_id, update_user_idに実行ユーザーIDを記録

変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-20	初版作成	Development Team